Goal. The goal of this work is prompt optimization for few-shot relation extraction, consisting of two components: *instruction optimization* and *example optimization*. Instruction optimization focuses on improving the task instructions in the prompt, while example optimization selects an optimal set of k input-output demonstrations. In our pipeline, instruction optimization is performed first, followed by example optimization.

Prompt Structure. In earlier experiments, we optimized the entire prompt jointly, including instructions, input placeholders, and answer constraints. In the current setup, we decompose the prompt into three parts:

1. **Instruction**: task description and guidance,
2. **Answer Instruction**: output constraints (e.g., answer format),
3. **Input**: relation name, support sentence, and query sentence.

During optimization, only the **Instruction** component is mutated and optimized. At inference time, the full prompt (instruction, answer instruction, and input) is provided. After instruction optimization is complete, we plan to prepend a set of optimized examples before the input for few-shot evaluation.

Baseline Prompt. A representative baseline prompt is shown below:

```
‘‘‘
You are given a relation name, a description of the relation in brackets,
a support sentence that exemplifies the relation, and a query sentence.

A relation connects the Subject and the Object entities. The Subject and
the Object entities are indicated with subject and object tags, respectively.
You need to decide whether the relation holds between the Subject and the
Object in the query sentence.

If the relation holds between the Subject and Object in the query sentence,
answer "yes"; otherwise, answer "no".

Output only "yes" or "no" as answer, with no explanation or additional text.

==
Relation name: "per:alternate_names" (a person’s alternate names)

Support Sentence: In high school and at Southern Methodist University, where,
already known as <object>Dandy Don</object>, <subject>Meredith</subject>
became an all-American.

Query Sentence: <subject>Escada</subject>, which employs around
<object>2,300</object> people worldwide, was forced to file for insolvency
in mid August.

Answer:
"""
```

Instruction Optimization Strategies. I am currently evaluating the following prompt optimization approaches:

- **Optimization without feedback**: the LLM updates the instruction to improve generalization without access to past correct or incorrect predictions.
- **Optimization with feedback**: the LLM updates the instruction using feedback from three past instances. I considered three variants: (i) only incorrect cases, (ii) only correct cases, and (iii) a mix of correct and incorrect cases.
- **Optimization with traces**: during mutation, the LLM is provided with up to two parent prompts along with their validation scores, and is asked to generate a new instruction by analyzing how prompt changes affected performance. I explored two variants: providing full parent prompts, or providing the root prompt along with a sequence of edit operations (e.g., add, remove, or replace text) in the child prompts.
- **Mixed strategy**: at each iteration, one of the above optimization strategies is selected in a round-robin fashion.

Next Plans. I plan the following improvements:

- **Crossover**: in addition to mutation, we will generate child prompts by combining two parent prompts. We consider two strategies: 1. random parents selection, and 2. select a prompt followed by selecting a complementary prompt based on performance diversity.
- **Prompt ensemble consistency**: instead of relying on a single optimized instruction or example set, we will use K optimized prompts and perform majority voting over their predictions. While similar ideas have appeared in a prior work for example optimization, we can extend this to instruction optimization too.
- **Grouping of Errors**: as shown in Mayank's paper [1], this leads to faster convergence.
- **Search Strategy**: plan to use MCTS to improve the search.

Preliminary Results. Based on 20 optimization iterations, it is difficult to identify a consistently superior strategy. Each approach improves performance in different ways. Experiments so far have been conducted on smaller language models; results are provided in google spreadsheet.

References

[1] Mayank Singh, Vikas Yadav, and Eduardo Blanco. Error taxonomy-guided prompt optimization, 2026.